Legacy SQL Script Analysis Report

ACE Online - Sales & Inventory System

Date: 2026-03-25

Summary

This report analyzes the legacy SQL view scripts used in the current sales and inventory management system. It identifies 10 critical issues across performance, scalability, maintainability, and data integrity. Each issue includes severity assessment, impact analysis, and recommended improvements.

1. Current Architecture Overview

The current system uses a chain of 7+ PostgreSQL views to calculate real-time inventory (stockreal). The calculation formula is:

```
stockreal = ingreso + offset - venta - suspendido
```

View dependency chain:

```
codigos (base table)
  -> corregidodetails_id_2        (offset corrections)
  -> itemdetails_id_2             (sales summary)
  -> ingresodetails_id_2          (income summary)
  -> itemdetails_today_id_2       (today's sales)
  -> ingresodetails_today_id_2    (today's income)
  -> cobdetalles_id_2             (suspended/reserved)
     -> screendetails2_id_2       (combined view)
        -> screendetails2_id      (per sucursal)
        -> screendetails2_deposito_id_tmp
           -> screendetails2_deposito_id  (warehouse)
              -> screendetails2_deposito_total_id
        -> screendetails2_total_id (grand total)
```

Each query to any final view triggers the entire chain, causing massive computational overhead as data grows.

2. Critical Issues

CRITICAL

1. Store-specific Hardcoded Views (_id_2 suffix)

All view names contain '_id_2' suffix, indicating they are created specifically for store_id=2. This means for 300 tiendas, you would need to create and maintain 300 separate sets of views (approximately 3,600+ views total).

```
-- Current: One set per store
CREATE VIEW screendetails2_id_2 AS ...  -- store 2 only
CREATE VIEW screendetails2_id_3 AS ...  -- store 3 copy
CREATE VIEW screendetails2_id_4 AS ...  -- store 4 copy
-- ... x 300 stores = 3,600+ views
```

Recommendation:

Replace with parameterized functions or a single view that includes store_id as a column, filtered at query time with WHERE clause.

```
-- Improved: Single parameterized function
CREATE FUNCTION get_screen_details(p_store_id INT, p_sucursal INT)
RETURNS TABLE(...) AS $$ ... $$ LANGUAGE sql;
```

CRITICAL

2. 7-Layer Nested View Chain (Performance)

The system chains 7+ views where each view depends on the previous one. PostgreSQL must execute the ENTIRE chain for every query to the final view. With 300 stores x 20,000 products = 6M products, each query triggers:
- 6M rows through corregidodetails
- 6M rows through itemdetails (with GROUP BY)
- 6M rows through ingresodetails (with GROUP BY)
- Multiple LEFT JOINs across all intermediate results

Estimated query time: 10-30+ seconds per final view query.

Recommendation:

Use MATERIALIZED VIEWs with REFRESH CONCURRENTLY, or better yet, maintain a denormalized stock_summary table updated by triggers on INSERT/UPDATE/DELETE events.

CRITICAL

3. Stock Calculated via View Chain (Data Integrity)

Real-time stock is calculated as:
  stockreal = ingresosumcant + cntoffset - ventasumcant - suspendido_total

This approach has critical risks:
- Any missed record (ingreso, venta, corregido) corrupts stock permanently
- No audit trail or reconciliation mechanism
- Race conditions: two simultaneous sales can both read same stock
- No constraint to prevent negative stock

Recommendation:

Maintain a 'current_stock' column updated atomically via triggers or application-level transactions with row-level locking (SELECT ... FOR UPDATE). Use stock_movements table as the audit log.

```
-- Atomic stock update with race condition prevention
UPDATE products SET stock = stock - :qty
WHERE id = :product_id AND stock >= :qty;
-- Returns 0 rows affected if insufficient stock
```

CRITICAL

4. No Index Strategy

The script creates views with heavy JOIN and WHERE conditions but defines no indexes. Critical missing indexes:

- codigos.borrado (used in every WHERE clause)
- vdetalle.ref_id_codigo + borrado + bfallado + bmovido (composite)
- vdetalle.fecha1 (date filtering for today's views)
- ingresos.ref_id_codigo + borrado + fecha (composite)
- cobdetalles.ref_id_codigo + borrado
- corregidos.ref_id_codigo + borrado

Without these indexes, PostgreSQL performs sequential scans on every table for every query. With 300K+ daily events, this becomes catastrophic.

CRITICAL

5. No Partitioning for High-Volume Tables

Tables like vdetalle (sales detail) and ingresos (income) grow by hundreds of thousands of records daily. The views filter by current_date but scan the ENTIRE table to find today's records.

Recommendation:

Implement range partitioning by date (monthly or weekly) on vdetalle and ingresos. This allows PostgreSQL to scan only the current partition for today's queries.

```
-- Partitioned table example
CREATE TABLE vdetalle (
  id BIGSERIAL, fecha1 DATE, ...
) PARTITION BY RANGE (fecha1);

CREATE TABLE vdetalle_2026_03
  PARTITION OF vdetalle
  FOR VALUES FROM ('2026-03-01') TO ('2026-04-01');
```

3. Structural Issues

WARNING

6. Inconsistent Column Naming

Current naming uses cryptic abbreviations that are hard to understand:

```
cant1, cant3        -> What is cant2? Why not 'quantity_sold', 'quantity_received'?
pre1~pre5           -> What are these 5 prices? No documentation
bfallado, bmovido   -> Boolean prefixed with 'b' (Hungarian notation)
fecha1, fecha       -> Multiple date columns with unclear purpose
kk                  -> Alias for cntoffset in final view (meaningless name)
nlocal              -> Same as sucursal? Confusing duplication
```

Recommendation:

Adopt clear, descriptive column names: quantity_sold, quantity_received, price_retail, price_wholesale, is_defective, is_transferred, sale_date, etc.

WARNING

7. Repetitive Soft Delete Pattern

Every JOIN and WHERE clause repeats 'borrado is false'. This is error-prone (forgetting it in one place corrupts results) and adds overhead.

```
-- Current: repeated in every query
WHERE c.borrado is false
AND v1.borrado is false
AND bfallado is false
AND bmovido is false
```

Recommendation:

Option A: Create base views that pre-filter deleted records.

Option B: Use Row-Level Security (RLS) policies.

Option C: Use partial indexes on (borrado = false).

```
-- Partial index: only indexes active records
CREATE INDEX idx_vdetalle_active
ON vdetalle (ref_id_codigo, fecha1)
WHERE borrado = false AND bfallado = false AND bmovido = false;
```

WARNING

8. Hardcoded Sucursal Defaults

COALESCE(sucursal, 1) and COALESCE(nlocal, 1) hardcode branch 1 as default. This masks NULL data issues and can assign stock to the wrong branch silently.

WARNING

9. Complex View Dependencies

The DROP/CREATE sequence must be executed in exact order. Any change to a base view requires dropping and recreating all dependent views. With 300 stores, schema changes become extremely risky (3,600+ views to update).

INFO

10. Debug Queries in Production Script

Multiple commented-out SELECT statements throughout the script suggest ad-hoc debugging. These should be removed from production scripts and maintained separately in a development/testing environment.

4. Scalability Impact Projection

Estimated data volume at scale:

Metric	Current (1 store)	Scale (300 stores)	Growth/Year
Products	20,000	6,000,000	+20%
Variants	100,000	60,000,000	+20%
Daily Events	1,000	300,000	+30%
Annual Records	365,000	109,500,000	+30%
Views Created	12	3,600+	Linear
Query Time (est.)	< 1s	10-30s+	Exponential

The current architecture has O(n*m) complexity where n=products and m=events. At 300 stores, every stock query scans ~60M product records and joins with ~100M+ event records. This is unsustainable without fundamental changes.

5. Recommended Architecture

Option A: Denormalized Stock Table + Triggers (Recommended)

Maintain a real-time stock_summary table updated atomically by triggers. This eliminates the need for view chains entirely.

```
-- Stock summary table (replaces all views)
CREATE TABLE stock_summary (
  product_id    BIGINT REFERENCES products(id),
  branch_id     INT REFERENCES branches(id),
  total_income  INT DEFAULT 0,
  total_sold    INT DEFAULT 0,
  total_reserved INT DEFAULT 0,
  total_offset  INT DEFAULT 0,
  current_stock INT GENERATED ALWAYS AS
    (total_income + total_offset - total_sold - total_reserved) STORED,
  last_income_date DATE,
  last_sale_date   DATE,
  PRIMARY KEY (product_id, branch_id)
);

-- Trigger on sale insert
CREATE FUNCTION update_stock_on_sale() RETURNS TRIGGER AS $$
BEGIN
  UPDATE stock_summary
  SET total_sold = total_sold + NEW.quantity,
      last_sale_date = NEW.sale_date
  WHERE product_id = NEW.product_id
    AND branch_id = NEW.branch_id;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

Benefits: O(1) stock lookup, no view chain, atomic updates, race condition safe with row-level locks.

Option B: Materialized Views with Periodic Refresh

If real-time accuracy is not required (e.g., stock checked every 5 minutes), convert views to MATERIALIZED VIEWs and refresh them periodically.

```
CREATE MATERIALIZED VIEW stock_mv AS
  SELECT ... (current view logic) ...;

CREATE UNIQUE INDEX ON stock_mv (product_id, branch_id);

-- Refresh every 5 minutes (no lock)
REFRESH MATERIALIZED VIEW CONCURRENTLY stock_mv;
```

Less ideal than Option A but simpler to implement as a migration path.

6. Migration Priority

1. IMMEDIATE

Action: Add indexes on frequently filtered columns (borrado, ref_id_codigo, fecha)

Impact: No schema change, 10-100x query speedup, zero downtime

2. SHORT-TERM

Action: Replace hardcoded _id_2 views with parameterized functions

Impact: Eliminates 3,600+ view maintenance, enables multi-store

3. MEDIUM-TERM

Action: Implement stock_summary table with triggers

Impact: O(1) stock lookup, eliminates view chain entirely

4. MEDIUM-TERM

Action: Add table partitioning on date-heavy tables

Impact: Dramatic speedup for date-range queries

5. LONG-TERM

Action: Migrate to normalized schema with clear naming

Impact: Improved maintainability and onboarding

7. Conclusion

The current view-based architecture works at small scale (1-2 stores) but will face severe performance and maintenance challenges at 300 stores. The most impactful improvement is replacing the view chain with a denormalized stock_summary table maintained by database triggers. Combined with proper indexing and partitioning, this approach can handle the projected scale of 6M+ products and 100M+ annual events while maintaining sub-second query times.